

INSTITUTE OF ENGINEERING
ADVANCED COLLEGE OF ENGINEERING AND MANAGEMENT
KALANKI, KATHMANDU
(AFFILIATED TO TRIBHUVAN UNIVERSITY)



ADVANCED COLLEGE
OF ENGINEERING AND MANAGEMENT

LAB REPORT
SUBJECT: Distributed System
LAB NO: 1

Submitted By:

Name: Himal Joshi
Roll No: ACE079BCT028
Date: June 3, 2026

Submitted To:

**Department of Electronics and
Computer Engineering**

IMPLEMENTATION OF RPC (REMOTE PROCEDURE CALL)

OBJECTIVES:

- To develop a solid understanding of how Remote Procedure Calls (RPC) operate between a client process and a server process across a network.
- To design and build a functional RPC mechanism in Python that exposes arithmetic operations (addition and subtraction) through a Pyro4-based client–server architecture.

THEORY:

Remote Procedure Call (RPC) is a distributed computing abstraction that lets one program invoke a function residing in a different process or on a remote machine, using the same syntax as an ordinary local function call. The underlying network details—socket creation, message framing, and transport management—are entirely hidden from the programmer, making distributed code easier to write and reason about.

RPC maps traditional procedural programming onto distributed environments, supporting modular service decomposition and well-defined client–server boundaries. It is extensively used in microservices and service-oriented architectures for synchronous inter-service communication.

Concrete implementations depend on four pillars:

- (1) stubs acting as local proxies on both sides.
- (2) a serialization scheme that encodes arguments and return values in a platform-neutral format.
- (3) a transport layer that delivers request and response messages reliably.
- (4) design choices around call semantics, failure handling, and interface versioning.

Working Principle

The RPC lifecycle proceeds through five well-defined phases:

- Client call → Client stub: The client invokes a local proxy (stub) that mirrors the remote function's signature, creating the illusion of a local call.
- Marshalling: The stub encodes function arguments into a portable byte stream suitable for network transmission.
- Transport: The encoded message is delivered to the server over a communication protocol—commonly TCP/IP; modern frameworks such as gRPC use HTTP/2 for multiplexing.
- Server stub → Unmarshalling: The server-side stub receives the byte stream, decodes it back into native types, and dispatches the call to the actual function implementation.
- Execution and response: The server runs the function, encodes the result (or error), and sends it back; the client decodes the response and returns the value to the caller as if it had been computed locally.

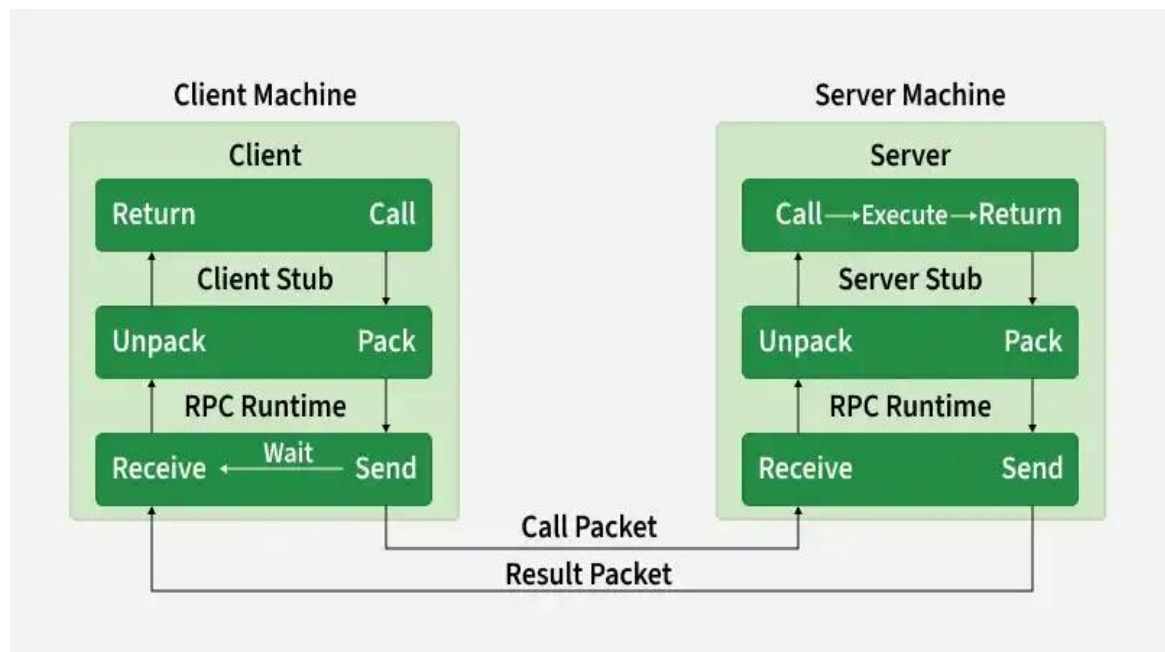


Figure 1: Block Diagram of RPC Implementation

Marshalling and Unmarshalling

Marshalling converts in-memory procedure arguments into a portable byte stream that can traverse the network. It ensures that complex data structures, primitive types, and metadata are represented consistently regardless of the underlying hardware architecture or operating system.

Unmarshalling performs the reverse: on the receiving end it reconstructs the original arguments and types from the byte stream so that the callee can operate directly on native objects. Both processes are symmetrical, and their correctness is fundamental to ensuring that distributed calls behave identically to local calls.

Communication Protocol

The communication protocol defines how marshalled messages are transported and delimited across the network. For a laboratory environment, TCP with a length-prefix framing scheme is a practical choice because it gives the receiver precise message-boundary information. Production-grade systems typically migrate to HTTP/2 or gRPC to gain multiplexing, flow control, and standardized headers. The protocol selection also governs call semantics (blocking vs. asynchronous), retry behavior, and the correlation of responses with their originating requests.

Middleware and Stubs

Middleware provides the infrastructure that makes RPC ergonomic to use. It generates or supplies client and server stubs, manages serialization and network I/O, and typically offers additional services such as service discovery, structured logging, and security hooks.

Stubs insulate application code from the complexities of marshalling and transport. From the programmer's perspective, writing `result = add(a, b)`

triggers a remote invocation, but all the underlying mechanics—argument encoding, socket communication, response decoding—are handled transparently by the middleware layer.

Pyro and Daemon

Pyro4 (Python Remote Objects) is a high-level RPC library that exposes ordinary Python objects to remote callers through auto-generated proxies. It automates proxy creation, object registration, and serialization (using pickle by default), allowing developers to focus on application logic rather than network plumbing.

A Pyro4 daemon is the server process that hosts registered objects and listens for incoming client connections. When a request arrives, the daemon unmarshals the arguments, dispatches the call to the appropriate registered object, and returns the serialized result. While Pyro4 is excellent for rapid prototyping in homogeneous Python environments, its default use of pickle introduces security risks with untrusted clients, and it lacks built-in cross-language interoperability.

SOURCE CODE:

1.server.py

```
1  import Pyro4
2
3  @Pyro4.expose
4  class Calculator(object):
5      def add(self, x, y):
6          return x + y
7
8      def subtract(self, x, y):
9          return x - y
10
11 # Create a Pyro4 daemon
12 daemon = Pyro4.Daemon()
13 uri = daemon.register(Calculator)
14
15 # Print the URI
16 print("Server URI:", uri)
17
18 # Start the server
19 print("Calculator server started. Press Ctrl+C to exit.")
20
21 try:
22     daemon.requestLoop()
23 except KeyboardInterrupt:
24     print("\nExiting Calculator server.")
25     daemon.shutdown()
```

2.client.py

```
1  import Pyro4
2  # Prompt the user to enter the URI of the server
3  uri = input("Enter the URI of the server: ")
4
5  # Connect to the server
6  calc = Pyro4.Proxy(uri)
7
8  # Function to perform addition
9  def perform_addition():
10     x = float(input("Enter the first number: "))
11     y = float(input("Enter the second number: "))
12     result = calc.add(x, y)
13     print("Result of addition:", result)
14
15 # Function to perform subtraction
16 def perform_subtraction():
17     x = float(input("Enter the first number: "))
18     y = float(input("Enter the second number: "))
19     result = calc.subtract(x, y)
20     print("Result of subtraction:", result)
21
22 # Interactive menu
23 while True:
24     print("\nChoose an operation:")
25     print("1. Addition")
26     print("2. Subtraction")
27     print("3. Exit")
28
29     choice = input("Enter your choice (1/2/3): ")
30     if choice == "1":
31         perform_addition()
32     elif choice == "2":
33         perform_subtraction()
34     elif choice == "3":
35         print("Exiting client.")
36         break
37     else:
38         print("Invalid choice. Please enter 1, 2, or 3.")
```

Output:

a. Server.py

```
python server.py
Server URI: PYRO:
PYRO:obj_eeb175f3a9a9462e9e9628a197dfd1b0@localhost:61749
Calculator server started. Press Ctrl+C to exit.
```

b. Client.py

```
python client.py
Enter the URI of the server: PYRO:
PYRO:obj_eeb175f3a9a9462e9e9628a197dfd1b0@localhost:61749
Choose an operation:
1. Addition
2. Subtraction
3. Exit
Enter your choice (1/2/3): 1
Enter the first number: 4
Enter the second number: 4
Result of addition: 8.0

Choose an operation:
1. Addition
2. Subtraction
3. Exit
Enter your choice (1/2/3): 2
Enter the first number: 5
Enter the second number: 1
Result of subtraction: 4.0

Choose an operation:
1. Addition
2. Subtraction
3. Exit
Enter your choice (1/2/3): 3
Exiting client.
```

DISCUSSION

This laboratory exercise used the Pyro4 library to implement Remote Procedure Call in Python, demonstrating the foundational concepts of distributed systems communication. A client–server architecture was adopted in which the client invoked arithmetic methods—add and subtract—that were physically hosted and executed on the server. Pyro4 simplified the RPC implementation considerably: it handled all networking, serialization, and remote-dispatch machinery automatically, allowing the application layer to remain clean and readable. Key distributed-systems concepts explored in the lab included remote object communication, service registration via the Pyro4 daemon, and network transparency—the property by which a remote call is indistinguishable from a local one at the source-code level. The Pyro4 name server was also examined as a mechanism for dynamic URI resolution, removing the need for hard-coded addresses. Several practical limitations of this implementation merit consideration. Manual URI entry at the client prompt is fragile and error-prone; in production a name server or service registry would resolve this. The code lacks input validation and exception handling around remote calls, making it vulnerable to crashes from malformed inputs or network interruptions. Pyro4's default serialization via pickle poses a security risk when communicating with untrusted clients, since pickle can execute arbitrary Python code on deserialization. Finally, the synchronous blocking call model means that a slow or unresponsive server would stall the client indefinitely; asynchronous or time-bounded calls would be necessary in any real deployment.

CONCLUSION

The laboratory successfully demonstrated the RPC paradigm using Pyro4, validating the core concepts of stub-based invocation, daemon-driven dispatch, and transparent remote execution. The implemented Calculator service correctly performed both addition and subtraction across separate client and server processes, confirming that the end-to-end RPC mechanism functioned as intended.

While the implementation is well-suited for educational purposes and rapid prototyping, transitioning it to a production environment would require several hardening steps: integrating a service discovery mechanism, adding robust input validation and structured error handling, replacing pickle with a safer serialization format such as JSON or Protocol Buffers, introducing TLS-encrypted transport for confidentiality and integrity, and adopting asynchronous call semantics to improve responsiveness and scalability.